Exercices C advanced

Structures

Assume the following structure

```
typedef struct
{
  int16_t speed;           // the car speed
  uint16_t RPM;            // the car motor RPM
  uint8_t gear;            // the engaged gear
}car_t;
car_t myCar;               // a car
```

1. Write a function who prints the speed and the RPM of myCar given as parameter.
2. Write a function who increments of 1 the speed of myCar given as parameter.
3. Write a function who copy one car to another, both given as parameter.
4. Write a function who returns the absolute speed of a given car as parameter.

Unions / bitfields

1. Write an union who offers access to ADCON0 with all bits and CHS group.

REGISTER 22-1: ADCON0: A/D CONTROL REGISTER 0

R/W-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
ADCAL	—	CHS3	CHS2	CHS1	CHS0	GO/$\overline{\text{DONE}}$	ADON
bit 7							bit 0

2. Assume the following union

```
typedef union
{
  car_t car;                    // the car struct
  uint8_t byte[sizeof(car_t)];  // the car by bytes
}carunion_t;
carunion_t myCar;               // a car
```

Swap the MSB / LSB of the RPM information with the byte access.

Swap the MSB / LSB of the RPM information with the RPM access.

Pointers

Assume the following structure

```
typedef struct user
{
  uint8_t name[20];         // user name
  struct user * next;      // the pointer to the next user
}user_t;
user_t * listP=NULL;        // the user list pointer
```

Write a function who takes the list Pointer and a name as argument to insert this user in the list:

1. Allocate the space for a new user.
2. Then, insert the name given as parameter (array).
3. Then, link the list to listP. The new user next will point to NULL.

Write another function who prints all the users' names in the list listP given as argument.

Write one more function who takes the list Pointer and a name as argument and delete this user in the list listP.

1. Find the users' name in the list.
2. Link correction (bypass this deleted link).
3. Release space of the deleted user.
4. Return 0 if user deleted, else return 1.

Modify your functions, if necessary, to be full safe (caution to pointers, arrays size, heap, …).